

Closed-Loop Gait Garment — Build Guide

Wireless version · CPX → ESP32-S3 mini → laptop · ENGF0031 Scenario 2

[START HERE](#)

1 How it fits together

One accelerometer in, one vibration motor out. The Circuit Playground Express (CPX) senses gait and drives the cue; an **ESP32-S3 mini** is a transparent UART⇌Wi-Fi bridge; the **laptop** runs the freeze-detection CNN and the dashboard. The return path (cue command → motor) is what makes it **closed-loop**, not just a logger.



Who dials whom: the laptop is the TCP **server** (binds 0.0.0.0:8765); the ESP32 is the **client** and connects out to the laptop's IP. A phone hotspot supplies the network. Same byte protocol as the old USB cable — only the wire changed, so the CNN and dashboard are unchanged.

2 Wiring & pin connections

CPX ⇌ ESP32-S3 mini — the UART link (3 wires)

Signal	CPX pad		ESP32-S3 GPIO	S3 pad	Notes
Accel data (up)	A7 · Serial1 TX	→	GPIO5 · RX2	D4	CPX transmits, ESP32 receives
Cue 'C'/'S' (down)	A6 · Serial1 RX	←	GPIO6 · TX2	D5	ESP32 transmits, CPX receives
Ground	GND	—	GND	GND	Common ground — mandatory

TX and RX cross over (one board's TX → the other's RX). Both boards are 3.3 V logic, so the lines connect **directly** — **no level shifter**. **Power each board from its own 5 V** (USB power bank / battery); share **only** the ground — do **not** also link the two 3.3 V rails.

Other ESP32 modules: the S3 has a GPIO matrix, so Serial2 can route to GPIO5/6 (pads D4/D5). On a **WROOM DevKitC** use GPIO16/17 instead; on a WROVER (PSRAM uses 16/17) pick two free GPIOs and edit RXD2/TXD2 in the sketch. The ESP32 status LED (GPIO2) is **solid** when linked to the laptop, **blinking while connecting**.

Vibration motor — on the CPX (pad A1, via an NPN transistor)

From		To
A1	→ 1 kΩ →	transistor base
motor (+)	—	VOUT (3.3 V)
motor (-)	—	collector
emitter	—	GND

Why a transistor? A CPX pad can't source the ~90 mA a motor needs, so A1 just switches an NPN (2N2222 / BC547).

- Add a **1N4148 flyback diode** across the motor (cathode → VOUT).
- **Avoid pad A0** — it is the onboard speaker / DAC pin.
- The 10 NeoPixels mirror the cue (red pulse) for the demo.

3 Flash & run

a • Flash the CPX

1. Arduino IDE: install **Adafruit SAMD Boards** + the **Adafruit Circuit Playground** library.
2. Open `firmware/cpx_fog_streamer/cpx_fog_streamer.ino`.
3. Board = **Adafruit Circuit Playground Express** → Upload.

b • Flash the ESP32-S3

1. Install **esp32 by Espressif** boards.
2. Open `firmware/esp32_wifi_bridge/esp32_wifi_bridge.ino` and set `WIFI_SSID`, `WIFI_PASS` (your hotspot) and `PI_HOST` = the laptop's IP.
3. Board = **ESP32S3 Dev Module, USB CDC On Boot = Enabled** → Upload.

c • Run on the laptop

Find the laptop IP on the hotspot (put it in `PI_HOST`):

```
ipconfig getifaddr en0 → e.g. 172.20.10.4
```

Then, in the `pi/` folder:

```
pip install fastapi 'uvicorn[standard]' scipy numpy torch
```

```
python dashboard_server.py
```

Open `http://localhost:8000/` in a browser.

No hardware? Dry-run the whole pipeline from recorded data:

```
python dashboard_server.py --replay  
<daphnet>/S01R01.txt
```

Order on the day: phone hotspot on → laptop server running → power both boards. When the ESP32 LED goes **solid**, the link is up and the dashboard starts moving.

4 If it won't link

Symptom	Fix
ESP32 never sees the hotspot SSID	On iPhone, turn Maximize Compatibility ON (forces 2.4 GHz). Every ESP32 is 2.4 GHz only.
Wi-Fi joins, but the dashboard shows no data	Let macOS allow incoming TCP 8765 (Firewall), and check <code>PI_HOST</code> matches <code>ipconfig getifaddr en0</code> (172.20.10.x on an iPhone hotspot).
ESP32 LED blinks forever	Wrong <code>WIFI_SSID</code> / <code>PASS</code> , the laptop server isn't running, or <code>PI_HOST</code> is wrong.
Linked, but no <code>ax</code> , <code>ay</code> , <code>az</code> lines	<code>TX/RX</code> not crossed, or no common ground. Re-check the table in §2.

Canonical code: `firmware/cpx_fog_streamer/ · firmware/esp32_wifi_bridge/ · pi/dashboard_server.py` Wi-Fi credentials live in the `.ino` only — never commit real ones.
`· pi/train_fog.py`